

Claude Code Alternatives: A Comprehensive Survey of AI Coding Agents in 2026 (Mid-2026 Refresh)

July 2026

Table of Contents

1	Introduction	2
2	The AI Coding Agent Taxonomy	3
3	CLI-Native Tools	4
3.1	Aider	4
3.2	OpenCode	7
3.3	Goose (by Block)	8
3.4	Gemini CLI (by Google)	10
3.5	Plandex	11
3.6	SWE-agent	12
3.7	OpenAI Codex CLI	13
3.8	Amp (Sourcegraph)	14
3.9	Deprecated and Unverified Tools	15
3.10	Mentat	15
3.11	Qwen Code and Kimi CLI	16
4	IDE Extensions and Hybrid Tools	16
4.1	Cline (formerly Claude Dev)	16
4.2	Continue.dev	18
5	Dedicated AI IDEs	20
5.1	Cursor	20
5.2	Windsurf (now Devin Desktop, Cognition AI)	21
5.3	Zed AI	22
5.4	Google Antigravity 2.0	23
6	Cloud and Web Platform Agents	24
6.1	OpenHands (formerly OpenDevin)	24
6.2	Devin (by Cognition AI)	25
7	Commercial AI Coding Assistants	26
7.1	GitHub Copilot with Agent Mode	26

7.2	Amazon Q Developer (formerly CodeWhisperer) — being sunset, see Kiro . . .	27
7.3	JetBrains AI Assistant	28
7.4	Tabnine	29
7.5	Supermaven — discontinued, folded into Cursor	29
8	Cross-Cutting Analysis	30
8.1	Provider Flexibility Analysis	30
8.1.1	OpenRouter Compatibility	30
8.1.2	Local LLM Support (Ollama / LM Studio)	34
8.2	Feature Comparison Matrix	37
9	* Unverified/lower-confidence — see the relevant tool’s section for sourcing caveats. † Gemini CLI’s 1M-token context window is a model-level feature, independent of the consumer free tier’s June 2026 deprecation (see Cost Analysis). ‡ Corrected from a bare “Yes”: Claude Code’s browser access is via MCP servers (e.g. Playwright) or WebFetch, not a built-in browser like Cline’s Puppeteer integration. § Corrected from a hedged claim — Goose’s MCP support is a confirmed, core architectural pillar (70+ extensions, 3,000+ compatible servers), not a disputed one. ¶ Gemini CLI’s consumer free tier ended June 18, 2026; only enterprise Code Assist tiers remain.	41
9.1	Extensibility Deep Dive	41
9.1.1	Claude Code’s Extension Architecture	41
9.1.2	How Alternatives Compare	42
9.1.3	The AGENTS.md Open Standard	43
9.1.4	What This Means for Migration	43
9.2	Cost Analysis	43
9.2.1	Defining “Heavy Usage”	44
9.2.2	API Cost Estimates (Direct Provider, as of July 2026)	44
9.2.3	Per-Tool Cost Summary	45
9.2.4	Cost Migration Strategy	53
10	Recommendations	54
10.1	If you want the closest CLI experience to Claude Code	54
10.2	If you want the best IDE-integrated alternative	54
10.3	If cost is the primary constraint	55
10.4	If MCP server investments are critical	55
10.5	If you need extensibility similar to Claude Code’s skills+hooks	56
10.6	Migration strategy	56
11	References	56

1 Introduction

Claude Code has established itself as one of the most capable agentic coding tools available: it runs entirely in the terminal, takes high-level natural-language instructions, autonomously

edits multiple files, executes shell commands, runs tests, and iterates until the task is done. Its extensibility system — skills, hooks, and MCP server support — allows deep customisation of its workflow. For heavy users, a Max plan (**\$100/month for Max 5x, \$200/month for Max 20x**) is good value relative to metered API pricing, but it is not unlimited: both tiers carry a 5-hour rolling session cap and a separate weekly cap (see Cost Analysis for the exact numbers). The plan landscape for AI tools changes rapidly, and a prudent engineer should understand the full landscape of alternatives before needing them.

This document surveys the entire landscape of AI coding agents available as of mid-2026: open-source CLI tools, IDE extensions, dedicated AI IDEs, cloud platform agents, and commercial assistants. For each, it covers architecture, provider flexibility, MCP/extensibility support, and realistic cost.

A note on methodology and provenance: This document was originally drafted from an AI model’s training-data snapshot (accurate as of approximately August 2025) and has since been fully refreshed as of July 2026 via live web research — every tool section, table, and cost figure was re-checked against current sources rather than carried forward unverified. Facts that were corrected or added during this July 2026 refresh are marked “verified 2026-07” (or a specific access date) at the point they’re introduced, with the corresponding source listed in the References section. Where a figure or claim carries no such marker, it should be read as inherited from the original snapshot and not independently re-checked this round — treat it with appropriate caution and verify before relying on it for a decision.

How to read this document: If you want the fastest path to a conclusion, jump to the [Feature Comparison Matrix](#), the [Provider Flexibility Analysis](#), and the [Recommendations](#). The deep-dive sections are there for when you need to evaluate a specific tool seriously.

This survey covers tools that were verifiable and actively maintained as of early 2026. It does not cover tools no longer in active development, purely GUI-based editors with no API or CLI surface, or general-purpose LLM interfaces that happen to accept code. Cloud IDE platforms (Replit, Gitpod, etc.) are out of scope unless they offer a dedicated coding-agent mode. Claims in this document are marked verified or refuted based on direct testing; where a claim could not be verified, it is flagged.

2 The AI Coding Agent Taxonomy

Before comparing tools, it helps to understand the four distinct categories that have emerged. This taxonomy comes from Artificial Analysis’s coding agent classification (verified):

CLI Tools run entirely in the terminal. They take instructions, edit files, run commands, and loop autonomously. This is the category Claude Code belongs to. Other members: Aider, OpenCode, Goose, Gemini CLI, Plandex, SWE-agent, Qwen Code, Kimi CLI, OpenAI Codex CLI, Amp (Sourcegraph, hybrid local/cloud), and GitHub Copilot CLI (distinct from the IDE-based Copilot covered under IDE Extensions below).

IDE Extensions augment an existing editor (primarily VS Code or JetBrains). They have full access to the editor’s language server, refactoring tools, and UI, but are less suitable for scripted or headless workflows. Members: Cline, Continue.dev, GitHub Copilot, Amazon Q Developer, Tabnine, JetBrains AI Assistant.

Dedicated AI IDEs are entire editors rebuilt around AI-first workflows. They typically fork VS Code and add deeper AI integration than an extension permits (Zed is the exception, built from scratch in Rust). Members: Cursor, Windsurf (now Devin Desktop, Cognition AI), Zed AI, AWS Kiro IDE, Google Antigravity 2.0.

Cloud Platform Agents run primarily in the cloud or a sandboxed environment (Docker). They expose a web UI or API and are designed for longer-running autonomous tasks, often with their own execution environments. Members: OpenHands, Devin, Manus, Jules, Genie.

A Claude Code user primarily cares about the CLI tools category, but the IDE and cloud categories contain tools capable enough to be worth understanding as alternatives — especially if your workflow includes time in an editor.

3 CLI-Native Tools

3.1 Aider

What it is: Aider is the most mature and architecturally similar open-source alternative to Claude Code. It is a terminal-based pair-programming tool that works directly with your existing git repository, takes natural-language instructions, autonomously edits multiple files, and commits changes.

Author and licence: Created by Paul Gauthier. MIT licence. Open source at github.com/Aider-AI/aider.

Community size: Approximately 47,100 GitHub stars (verified 2026-07, up from 45,300 in May 2026). This is among the largest in the open-source CLI coding agent category, though development cadence has slowed relative to faster-moving peers like OpenCode and Cline —

the last push to the main repo was May 22, 2026, versus daily pushes elsewhere. Worth watching as a trend rather than dismissing as a one-off snapshot.

Architecture: Aider is written in Python and uses the `litellm` library as its LLM abstraction layer, which means it can talk to virtually any LLM provider that `litellm` supports. The editing mechanism is based on unified diffs: the model is asked to produce a git-style diff, which Aider then applies to the working tree and automatically commits. This is a deliberate design choice — unified diffs are compact and less error-prone for the model to produce than rewriting entire files. Aider supports several edit formats depending on the task and model:

- **diff** — the default, produces unified diffs
- **whole** — the model rewrites the entire file (less efficient, sometimes more reliable for small files)
- **architect mode** — a two-step process where one model plans the changes and a second cheaper model applies them, reducing cost

Installation:

```
pip install aider-chat
# or with uv
uv tool install aider-chat
```

LLM provider support: Aider supports all major providers via `litellm`, including:

- Anthropic (Claude family — `claude-sonnet-5`, `claude-opus-4-8`, `claude-haiku-4-5-20251001`)
- OpenAI (`gpt-5.5`, `gpt-5.3-codex`)
- Google (`gemini-3.1-pro-preview` via Vertex AI or AI Studio)
- AWS Bedrock
- Azure OpenAI
- OpenRouter (verified — full setup documented at `aider.chat/docs/llms/openrouter.html`)
- Ollama for local models
- LM Studio
- Groq, Mistral, Cohere, and many others

OpenRouter setup (verified):

```
export OPENROUTER_API_KEY=sk-or-...
aider --model openrouter/anthropic/claude-sonnet-5
# or any OpenRouter-hosted model:
aider --model openrouter/deepseek/deepseek-v4-pro
aider --model openrouter/google/gemini-3.1-pro-preview
```

Local LLM setup (Ollama):

```
# Start Ollama with a coding model first
ollama pull qwen2.5-coder:32b

# Then run Aider
aider --model ollama/qwen2.5-coder:32b
```

Agentic capabilities: Aider can run shell commands via the `/run` command and in `--auto-run` mode will execute suggested commands automatically. It maintains a context of added files and can be instructed to add more mid-session. It supports `/web` for fetching URLs into context and can integrate with test runners. The workflow is: add files to context → give instruction → model proposes diffs → Aider applies and commits → repeat.

Git integration: Every accepted change is automatically committed. This is tightly integrated — you can see the full history of AI changes in `git log`. There is also an `--auto-commits` flag to control this behaviour.

Shell execution: Via `/run <command>` or `--auto-run`. Aider can also be configured with `--test-cmd` to run tests automatically after each change.

MCP support: Still absent as of mid-2026. RFC #4506 proposing native MCP support remains open, and an exploratory PR (#3937) was closed without shipping. This is a meaningful and persistent gap compared to Claude Code — eleven months on from the original assessment, the gap has not closed.

Extensibility: Aider has limited plugin architecture compared to Claude Code. Configuration is via `.aider.conf.yml` and environment variables. There is no equivalent to Claude Code's skills or hooks system. What it lacks in extensibility it compensates for in simplicity and reliability.

Modes:

- **code** — the default mode, edits files
- **architect** — planning model + editing model split
- **ask** — ask questions about code without editing
- **help** — help with Aider itself

Cost model: Open source, free to use. You pay only for the LLM API calls you make. With Claude Sonnet 5 via the Anthropic API, a typical coding session might cost \$0.50–\$3.00 depending on context size and number of changes. With OpenRouter you can route to cheaper models.

Strengths: - Most CLI-native and architecturally similar to Claude Code of all open-source options - Excellent git integration — every change is tracked - Wide provider support via litellm - OpenRouter support is well-documented and works reliably - Mature, battle-tested codebase with high benchmark performance - Supports local LLMs via Ollama - Very low overhead — just Python and an API key

Weaknesses: - No MCP support, and no committed timeline for one (RFC open, prior PR

abandoned) - No skills/hooks system — less extensible than Claude Code - Context management is manual (you explicitly `/add` files) - Less capable at long multi-step autonomous tasks compared to Claude Code - No built-in web search or browser use - Development cadence has slowed relative to faster-moving competitors

3.2 OpenCode

What it is: OpenCode is a terminal UI (TUI) coding agent written in Go, presenting a rich interactive terminal interface while supporting a very wide range of LLM providers.

Author and licence: Originally built by the SST team; the project rebranded in early 2026 under a new organisation, Anomaly Inc., and the repository moved from `sst/opencode` to `anomalyco/opencode`. Open source.

Community size and activity: 183,000 GitHub stars and 7.5 million+ monthly active users (verified 2026-07) — a step change from the sub-100k star count of the prior assessment, alongside continued rapid releases (v1.17.14 as of July 6, 2026) and 75+ connected providers. This is one of the most actively maintained and now most widely used tools in the space.

Architecture: OpenCode is a Go binary with a full TUI (terminal user interface) built using the Bubble Tea framework. It integrates with the AI SDK and Models.dev for LLM provider abstraction. It also supports Language Server Protocol (LSP) for code intelligence, meaning it can provide type-aware, semantically accurate code context rather than just raw file contents. Multi-session support allows you to maintain separate contexts for different tasks.

Installation:

```
# via npm (cross-platform)
npm install -g opencode-ai
```

```
# or directly via binary release
curl -fsSL https://opencode.ai/install | bash
```

LLM provider support: OpenCode supports 75+ LLM endpoints (verified) including:

- OpenAI (ChatGPT subscription plans now work natively)
- Google (Gemini via AI Studio or Vertex)
- AWS Bedrock
- Azure OpenAI
- OpenRouter
- Ollama (local models)
- LM Studio
- Grok (xAI)

- And many more via the AI SDK / Models.dev integration

This is the broadest provider flexibility of any CLI tool in the category.

Important change — Anthropic access: In January 2026, OpenCode broke with Anthropic: Claude models are no longer bundled or accessible through a simplified login flow. Using Claude now requires supplying your own Anthropic API key directly, the same as any other provider. Over the same period, native ChatGPT subscription support was added, so the tool is no longer usefully described as “Claude-adjacent” — if anything, the balance now tilts toward OpenAI’s consumer plans for users who don’t want to manage a separate API key.

OpenRouter and Ollama: Both confirmed as supported providers. Configuration is via a `~/.config/opencode/config.json` or project-level config.

Agentic capabilities: OpenCode can read and edit files, run shell commands, and iterate autonomously. The TUI provides a chat interface that feels more like a dedicated application than a REPL. LSP integration means it can navigate symbol definitions, find references, and understand project structure at the type level.

MCP support: OpenCode supports MCP servers. This is a significant advantage over Aider for users who have invested in an MCP ecosystem.

Cost model: Open source, free to use. API costs only — for Claude models this now means a directly billed Anthropic API key rather than any bundled access.

Strengths: - Broadest LLM provider support of any CLI tool (75+ endpoints verified) - Rich TUI with a more polished interactive experience than most CLI tools - LSP integration for semantic code understanding - MCP support - Extremely active development, now with a large user base (7.5M+ MAU) - Built in Go — fast, single binary, no Python dependency hell - Native ChatGPT subscription support

Weaknesses: - Relatively new compared to Aider — less battle-tested - TUI approach is slightly less scriptable than a pure REPL like Aider - Claude access now requires a raw Anthropic API key — no bundled access, unlike before the January 2026 split - Org/rebrand churn (`sst/opencode` → `anomalyco/opencode`) means older documentation and links may be stale

3.3 Goose (by Block)

What it is: Goose is an open-source, on-device AI coding agent developed by Block (formerly Square, the company associated with Jack Dorsey). It is available as a CLI, a desktop

application, and an API. Its primary differentiator is an extension system built natively on MCP.

Author and licence: Developed by Block, Inc. Apache 2.0 licence. Open source at github.com/block/goose.

Architecture: Goose runs on-device, meaning all execution happens locally on your machine (the LLM calls go to wherever you configure, including local Ollama). Its extension system is built directly on MCP rather than a Goose-proprietary format — MCP is a core architectural pillar of the tool, not a bolted-on integration. Goose ships 70+ documented extensions and is compatible with 3,000+ MCP servers from the broader ecosystem, so existing MCP investments carry over directly.

Installation:

```
# macOS  
brew install block/tap/goose
```

```
# or via install script  
curl -fsSL https://github.com/block/goose/releases/latest/download/install.sh | bash
```

LLM provider support: Goose is model-agnostic and supports multiple provider configurations simultaneously, with 15+ documented provider integrations including Anthropic, OpenAI, Google, Groq, and Ollama. It does not lock you to a single provider and allows per-task model configuration.

Local LLM support: Confirmed support for Ollama — this is a core feature of the on-device design, not a partial or unverified claim.

Agentic capabilities: Goose can edit files, run shell commands, use its extension system to call external tools, and iterate autonomously on tasks. It has a particularly strong story for DevOps and infrastructure tasks.

MCP support: Confirmed and central to the architecture, not a hedged or disputed claim. Goose’s extension system is built on MCP, ships 70+ documented extensions, and is compatible with 3,000+ MCP servers. In December 2025, Block contributed Goose to the Linux Foundation’s new Agentic AI Foundation, co-stewarding alongside Anthropic’s MCP and OpenAI’s AGENTS.md — a further sign of Goose’s commitment to MCP as core infrastructure rather than a side feature.

Extensions: The extension system is the key differentiator, and it is MCP-native: extensions are effectively MCP servers, giving Goose access to the same growing ecosystem Claude Code and other MCP-compatible tools draw on.

Strengths: - Strong corporate backing from Block — less likely to be abandoned - On-device

execution philosophy — good for privacy - MCP-native extension system with 70+ documented extensions and 3,000+ compatible servers - Desktop app available for non-terminal users - Apache 2.0 licence - Model flexibility across 15+ providers - Co-steward of the Linux Foundation’s Agentic AI Foundation

Weaknesses: - Less pure CLI tool than Aider — the desktop app orientation means some rough edges in headless terminal use - Younger MCP ecosystem integration story than Claude Code’s, despite the architecture being sound

3.4 Gemini CLI (by Google)

What it is: Gemini CLI is Google’s open-source terminal AI agent, released in mid-2025. It is powered by the Gemini model family. Through mid-2026 it offered generous free-tier access for individual users; that consumer free tier has since ended (see below), materially changing its cost position relative to other CLI tools.

Author and licence: Google Cloud/Developer Products (not Google DeepMind, contrary to earlier reporting). Apache 2.0 licence. Open source at github.com/google-gemini/gemini-cli.

Community size: Gemini CLI saw extremely rapid adoption after launch — it became one of the fastest-growing GitHub repositories in history within weeks of release.

Architecture: Gemini CLI is a Node.js-based CLI agent. It uses Gemini’s Pro-tier model by default, which provides a very large context window — among the largest of any CLI coding tool. This means you can load entire codebases into context rather than managing file additions manually.

Installation:

```
npm install -g @google/gemini-cli  
gemini
```

LLM provider support: Gemini CLI is primarily designed for Google’s Gemini models (gemini-3.1-pro-preview, gemini-3.5-flash). It is not model-agnostic in the same way as Aider or OpenCode.

Free tier — deprecated June 18, 2026: This is the most consequential change in this refresh. Google discontinued consumer-tier free access on June 18, 2026: Gemini Code Assist for Individuals, Google AI Pro, and Google AI Ultra all stopped serving free requests. Only enterprise tiers (Gemini Code Assist Standard/Enterprise) continue to be offered. Gemini CLI should no longer be recommended as a zero-cost option for individual developers — that recommendation is now obsolete, not merely dated. Consumer users who relied on the free tier are being directed by Google toward the new **Google Antigravity** platform instead (see the Dedicated AI IDEs section), which is free for individuals and model-agnostic.

MCP support: Gemini CLI supports MCP servers, making it one of the CLI tools with confirmed MCP integration alongside Claude Code and Goose. Configuration is via a `~/.gemini/settings.json` file.

Extensions: Gemini CLI has an extensions mechanism that allows adding capabilities beyond the default file editing and shell execution.

Agentic capabilities: Gemini CLI can read and edit files, run shell commands, search the web, and iterate autonomously. Its large context window means it is uniquely capable at tasks involving entire-codebase understanding.

Cost model: As of June 18, 2026, individual/consumer users no longer have a free tier — standard Gemini API pricing applies for the Pro and Flash models, or an enterprise Gemini Code Assist subscription is required. Budget-conscious individuals should look to Google Antigravity or a different tool’s free/local tier instead.

Strengths: - Large context window — can load entire codebases - MCP support - Open source (Apache 2.0) - Google backing means long-term support likely - Very active development

Weaknesses: - Locked to Gemini models — no OpenRouter or third-party model support - No longer free for individual/consumer use as of June 18, 2026 — a major change from its original positioning - Gemini models, while capable, may not match Claude’s code quality for your specific use cases - Less battle-tested than Aider - Node.js dependency

3.5 Plandex

What it is: Plandex is an open-source CLI agent with a distinctive planning-first approach. Rather than immediately executing changes, it plans multi-file, multi-step tasks and builds up pending changes for user review before applying.

Author and licence: Created by Dane Schneider (@danenania). MIT licence. Open source at github.com/plandex-ai/plandex. Approximately 15,500 GitHub stars, currently at v2.2.1.

Architecture: Plandex is written in Go and uses a client-server architecture. The server can be run locally or self-hosted on a cloud provider. Changes are accumulated in a “pending changes” buffer that you can review, modify, or reject before applying to the working tree. This makes Plandex the most review-oriented tool in the category — it assumes you want to understand what’s happening before it happens.

Installation:

```
# Install client
curl -sL https://plandex.ai/install.sh | bash

# Use cloud server (free tier available)
plandex sign-in

# Or self-host the server
docker-compose up -d # with the plandex-server repo
```

LLM provider support: Plandex supports OpenAI models directly and OpenAI-compatible APIs, which includes many providers. OpenRouter support via the OpenAI-compatible endpoint is possible.

Self-hosting: A key differentiator — you can run the entire Plandex server yourself, meaning no data leaves your infrastructure.

Agentic capabilities: Plandex is specifically designed for large, multi-step tasks: “implement this feature across 20 files.” Its planning mechanism handles long sequences of related changes better than tools that tackle each change independently.

Strengths: - Planning-first approach is excellent for large refactors - Self-hosting option for data sovereignty - Review buffer means less risk of unexpected changes - Good at long-horizon multi-file tasks

Weaknesses: - Planning overhead makes it slower for quick edits - Less interactive than Aider or OpenCode - Client-server architecture adds complexity - LLM provider support less broad than Aider/OpenCode - Smaller community

3.6 SWE-agent

What it is: SWE-agent is a research tool from Princeton NLP lab, designed primarily for automated software engineering on GitHub issues. It achieves high scores on the SWE-bench benchmark. It is less a daily-driver tool and more a demonstration of what autonomous agents can accomplish on discrete bug-fix tasks.

Author and licence: Princeton NLP group. MIT licence. github.com/SWE-agent/SWE-agent.

Architecture: SWE-agent uses a Docker-based sandboxed environment. The Agent-Computer Interface (ACI) pattern it introduces provides structured tools for file navigation, editing, and execution in a way that is particularly suited for benchmark tasks. SWE-agent v1.1.0 remains state-of-the-art on SWE-Bench as of February 2026 and is not abandoned, but active development in this lineage has shifted to a sibling project: **mini-swe-agent**, a ~100-line Python reimplement of the same core idea, which has grown to 5.4k stars and reaches roughly 65% on SWE-bench Verified. If you’re evaluating this family of tools today,

check mini-swe-agent as well as SWE-agent proper — the smaller project is where most current iteration is happening, even though the original benchmark numbers still stand on SWE-agent itself.

Intended use: Automated bug-fixing on GitHub issues, especially in batch/CI contexts. Not designed for interactive development sessions.

Strengths: - Extremely high benchmark performance - Docker sandboxing for safe execution
- Good for automated CI/CD pipelines

Weaknesses: - Not a daily driver — requires Docker, not interactive - Research-oriented design means rough UX edges - Slower than interactive tools - Not designed for the ad-hoc, exploratory coding sessions that Claude Code excels at - Primary development momentum has moved to mini-swe-agent; SWE-agent proper sees less active iteration

3.7 OpenAI Codex CLI

What it is: Codex CLI is OpenAI’s open-source terminal coding agent, open-sourced April 16, 2025. It is OpenAI’s direct answer to Claude Code and Gemini CLI: a terminal-native agent that reads and edits files, runs shell commands, and iterates autonomously, with tight integration into ChatGPT’s subscription plans.

Author and licence: OpenAI. Open source at github.com/openai/codex. Approximately 95,900 GitHub stars (July 2026); the codebase is 96.5% Rust.

Architecture: Codex CLI is a Rust binary, giving it a fast startup and low overhead compared to Node.js- or Python-based competitors. It defaults to OpenAI’s own hosted models but also supports local model backends.

Installation:

```
npm install -g @openai/codex  
# or via the standalone Rust binary release  
codex
```

Model support: The default model is **gpt-5-codex**, OpenAI’s flagship coding model; a budget variant, **gpt-5-codex-mini**, is available and gives roughly 4x the usage budget for lighter workloads. These are Codex’s own product-tier model names — distinct from the versioned **gpt-5.x** family used elsewhere in the OpenAI API (e.g. **gpt-5.5**, **gpt-5.4**) — so don’t conflate the two naming schemes when comparing pricing or capability.

Local model support: Codex CLI can run against local models via an `--oss` flag, which supports Ollama, LM Studio, and MLX backends. This gives it a genuine offline/local-only mode, something Gemini CLI lacks entirely.

Bundled pricing: Codex CLI usage is bundled into ChatGPT’s consumer and business plans rather than metered separately by default: Free, Go (\$8/mo), Plus (\$20/mo), Pro (\$100/mo), plus Business/Edu/Enterprise tiers. This mirrors Claude Code’s Max-plan bundling model more closely than a pay-per-token CLI tool like Aider.

Configuration: Config lives at `~/.codex/config.toml`, covering model selection, sandboxing behaviour, and approval policy for shell commands.

Agentic capabilities: Codex CLI can read and edit files, run shell commands with configurable approval levels, and iterate autonomously across multi-step tasks, broadly comparable in scope to Claude Code and Gemini CLI.

Strengths: - Backed by OpenAI’s flagship coding model, `gpt-5-codex`, with a cheaper `-mini` variant for budget-conscious usage - Fast, low-overhead Rust binary - Genuine local-model mode via `--oss` (Ollama, LM Studio, MLX) — not just a cloud-only tool - Bundled into existing ChatGPT subscriptions, so many users already have access with no incremental cost - Very large and fast-growing community (95.9k stars)

Weaknesses: - Best experience is tied to OpenAI’s own models and ChatGPT plans; using it as a purely model-agnostic tool is a secondary use case - Newer than Aider/OpenCode as a CLI-first product, despite OpenAI’s scale - Product-tier model naming (`gpt-5-codex`) is a separate namespace from the API’s versioned models, a frequent source of confusion when comparing pricing across tools

3.8 Amp (Sourcegraph)

What it is: Amp is Sourcegraph’s 2026 rebrand of its earlier Cody assistant, repositioned as a hybrid CLI-plus-cloud coding agent. It pairs a local CLI with cloud-hosted “Orbs” for offloading longer-running or parallel work, similar in spirit to Goose’s on-device-plus-extensions model but with a cloud execution tier built in from the start.

Author and licence: Sourcegraph.

Architecture: Amp is built on Sourcegraph’s existing code-intelligence infrastructure — cross-repo search, dependency analysis, and code-graph features Sourcegraph has developed for years as a code search company. This gives it a differentiated strength: understanding large, multi-repo codebases at a structural level, rather than relying solely on what fits in a model’s context window. The CLI handles local, interactive work; Orbs handle cloud-side, longer-running or parallelised tasks.

LLM provider support: Model-agnostic, with plugin and event-hook extensibility for

customising behaviour around model calls and tool use.

Cost model: Pay-as-you-go, with no markup applied for individual users over the underlying model cost. Sourcegraph has not published a full rate card for Amp as of this writing — treat exact pricing as an open question rather than assuming parity with any other tool’s numbers.

Strengths: - Deep code-intelligence heritage (cross-repo search, dependency graphs) not available in most other CLI tools - Hybrid local/cloud model gives a path to offloading long-running or parallel tasks without leaving the CLI workflow - Model-agnostic with extensibility via plugins/event hooks - No markup on individual pay-as-you-go pricing

Weaknesses: - Full pricing is not yet publicly documented — evaluate the actual cost before committing to it for heavy use - Newer product identity (rebrand from Cody in 2026) means less accumulated community track record under the Amp name specifically - Cloud “Orbs” component means it is not a purely local/offline tool in the way Aider or Goose can be

3.9 Deprecated and Unverified Tools

3.10 Mentat

What it is: Mentat was a Python-based CLI coding tool and an early entrant in the space, taking a conversational approach to code editing with explicit context management.

Status: Fully archived and read-only since January 7, 2025 — this is a firm end-of-life, not a gradual slowdown. Note for anyone searching for current status: “Mentat” is now also used by an unrelated GitHub bot product from Abante AI, which can make search results confusing — the archived coding CLI and the Abante AI bot are different products that happen to share a name.

Note: For new users, Aider is a better choice in the same category — more active, more features, larger community.

Roo Code (fork of Cline) — also deprecated: Roo Code, a fork of Cline created in 2024 to move faster on multi-agent workflows (its “Boomerang Tasks” feature) and broader model support, was archived on May 15, 2026, after peaking at roughly 24,300 stars and 1.55 million VS Code installs. The team pivoted to a cloud agent product, roomote.dev, on the view that the IDE is no longer the right locus for this kind of work. Most of Roo Code’s userbase has since moved to Cline itself (see the Cline section) or to Kilo Code, another Cline-derived fork. There is no live “current tool” entry for Roo Code in this document — by the time of this refresh it is itself historical.

3.11 Qwen Code and Kimi CLI

What they are: These are CLI coding agents from Chinese AI labs — Qwen Code from Alibaba’s Qwen team and Kimi CLI from Moonshot AI. Both are classified alongside Claude Code and Aider as CLI tools by Artificial Analysis’s taxonomy. Both are positioned as competitive alternatives particularly for users who want model diversity or prefer models strong at certain programming languages.

Provider flexibility: Qwen Code uses Qwen models (strong at code, particularly multilingual/non-English codebases; current coding-focused models include `qwen3-coder-next`, `qwen3-coder-plus`, and `qwen3-coder-480b`). Kimi CLI uses Kimi’s models with a very long context window.

Note: These tools were not covered by this round’s research refresh. Treat specific capability claims with appropriate scepticism until you test them directly.

4 IDE Extensions and Hybrid Tools

4.1 Cline (formerly Claude Dev)

What it is: Cline is an open-source autonomous coding agent that operates primarily as a VS Code extension. It is model-agnostic, OpenRouter-compatible, and one of the most capable agentic tools available for IDE-based workflows. It has since expanded well beyond VS Code and beyond the IDE entirely, moving it toward hybrid territory.

Author and licence: Open source at github.com/cline/cline. MIT licence.

Community size: Approximately 61,000-62,000 GitHub stars (mid-2026; the figure fluctuates within this band depending on snapshot date, not a material change from prior counts). This makes it one of the most popular tools in the entire AI coding agent space, not just this subcategory.

Platform reach: Beyond VS Code, Cline now runs in JetBrains IDEs, Cursor, Windsurf, Zed, and Neovim, plus a CLI preview for macOS and Linux — the clearest sign yet of the shift from “VS Code extension” to platform-agnostic agent.

Architecture: Cline runs as a VS Code extension and has deep access to the VS Code API — language servers, the file system, the integrated terminal, and the browser (via Puppeteer integration). It operates in two modes:

- **Plan mode** — Cline analyses the task and produces a plan before executing
- **Act mode** — Cline executes changes directly

This Plan/Act split allows you to review the strategy before any files are modified.

LLM provider support: Cline is model-agnostic. It supports:

- Anthropic (Claude family)
- OpenAI (GPT-4o, o1)
- Google (Gemini)
- AWS Bedrock
- Vertex AI
- OpenRouter (verified — listed on OpenRouter’s works-with-openrouter page)
- Ollama for local models
- LM Studio
- Any OpenAI-compatible API endpoint

OpenRouter setup:

In VS Code: Open Cline extension settings

API Provider: OpenRouter

API Key: sk-or-...

Model: anthropic/claude-sonnet-5 (or any OpenRouter model)

Agentic capabilities: Cline can:

- Read and edit any file in the workspace
- Execute shell commands in the integrated terminal
- Create and delete files/directories
- Use the browser (view pages, click, type, screenshot)
- Run searches across the codebase
- Read error output and iterate automatically

Browser use is a standout capability — Cline can actually open a browser, navigate to a URL, interact with a web page, and read the results. This enables tasks like: “go to this API docs page and implement the integration.”

MCP support: Cline supports MCP servers. Configuration is via the Cline settings JSON. This means your existing MCP server investments (filesystem servers, database servers, custom tools) are portable to Cline.

Extensibility: Beyond MCP, Cline has a system prompt customisation capability and supports `.clinerules` files (analogous to `.cursorrules` or Claude Code’s `CLAUDE.md`) for project-specific behaviour.

CLI mode: The standalone CLI preview (macOS/Linux) makes Cline usable outside any IDE. This is a recent addition and may not have full feature parity with the VS Code extension yet, but the trajectory is toward a full CLI-native experience.

Cost model: Open source, free. API costs only. Using OpenRouter you can route to cheaper models for lower-cost tasks.

Team note: OpenAI acqui-hired roughly seven of Cline’s core staff in early 2026. This was a talent acquisition, not a project acquisition — Cline itself remains independent, open source, and actively developed, though it’s worth knowing if you notice any shift in velocity.

Roo Code lineage: Roo Code forked from Cline in 2024 to push further on multi-agent workflows (Boomerang Tasks) and broader model support. It was archived on May 15, 2026, having peaked around 24,300 stars and 1.55M VS Code installs; most of its userbase returned to Cline, with the rest moving to Kilo Code.

Strengths: - Extremely capable agentic tool — one of the best - Model-agnostic with OpenRouter support verified - Browser use capability is unique among open-source tools - MCP support - Plan/Act mode for controlled execution - Very large and active community - Now reaches far beyond VS Code (JetBrains, Cursor, Windsurf, Zed, Neovim, CLI)

Weaknesses: - Still primarily an IDE-centric tool — the CLI preview trails the extension in feature parity - Browser use capability means it can have unintended side effects if not supervised - Less scriptable than pure CLI tools

4.2 Continue.dev

What it is: Continue.dev is an open-source AI coding assistant for VS Code and JetBrains IDEs. It combines autocomplete, chat, and agent capabilities in a single extension, with deep configurability via a YAML config file. Since September 2025 it also ships a headless CLI, so it is no longer purely IDE-integrated.

Author and licence: Open source at github.com/continuedev/continue. Apache 2.0 licence.

Architecture: Continue.dev is structured around three core features:

1. **Autocomplete** — inline code completions as you type
2. **Chat** — conversational interface about code, with codebase context
3. **Agent** — agentic mode that edits files based on instructions

Configuration lives in `~/.continue/config.yaml` and can specify different models for different features (e.g., a fast local model for autocomplete, a more capable cloud model for agent tasks).

CLI mode (cn): Continue.dev shipped a headless `cn` CLI in September 2025, offering both a TUI interactive mode and a scriptable/background mode (`cn -p "prompt"` for one-shot

invocation, plus async background jobs). This is a supplementary capability, not a pivot away from the IDE extension — Continue.dev is still primarily used as an IDE assistant, with the CLI extending it into scripts, CI, and terminal-first workflows.

LLM provider support: Continue.dev has one of the broadest provider lists:

- Anthropic, OpenAI, Google
- OpenRouter
- Ollama (confirmed at localhost:11434 by default, remote connections via **apiBase**)
- LM Studio, llama.cpp
- Groq, Together AI, Replicate
- Self-hosted models via any OpenAI-compatible endpoint

Ollama setup (verified):

```
# ~/.continue/config.yaml
models:
- title: "Local Qwen3 Coder"
  provider: ollama
  model: qwen3.5-9b
  # For remote Ollama:
  # apiBase: http://192.168.1.100:11434
```

Agent mode: Continue.dev’s agent mode allows it to make multi-file edits based on instructions. Tool support depends on the underlying model — models must support function/tool calling. Note: some models that claim tool support may not work reliably in agent mode (this was partially corroborated in the research, though the specific claim was refuted as too broad).

Extensibility: Beyond MCP, Continue.dev supports a block system where community-built extensions (“blocks”) add new context providers, tools, and model configurations. This is effectively a plugin marketplace for the IDE assistant workflow.

Strengths: - Best-in-class for IDE users who want full local LLM support - Works in both VS Code and JetBrains - Highly configurable via config.yaml - Autocomplete + chat + agent in one package - OpenRouter and Ollama support confirmed - Headless **cn** CLI extends it into scripts, CI, and terminal workflows - Open source, Apache 2.0

Weaknesses: - Still primarily IDE-integrated; the CLI is supplementary, not a terminal-first redesign - Agent mode quality depends heavily on the model’s tool-calling capabilities - Less capable at long agentic chains than Cline or Claude Code - JetBrains support is less mature than VS Code

5 Dedicated AI IDEs

5.1 Cursor

What it is: Cursor is a VS Code fork with deep AI integration baked into every layer of the editor. It has become one of the most widely adopted commercial AI coding tools, with a large user base and reported ARR in the hundreds of millions.

Company: Anysphere, Inc. Private company, significant VC backing.

Pricing:

- **Hobby** — Free — limited completions and requests per month
- **Pro** — \$20/month — usage-based credit pool (not a fixed request count)
- **Pro+** — \$60/month — larger credit pool for heavier usage
- **Ultra** — \$200/month — highest credit pool, priority access to frontier models
- **Teams** — \$40/user/month — team features, admin controls, SOC 2 compliance
- **Enterprise** — custom

Billing moved from a fixed “500 fast requests/month” allowance to a usage-based credit pool in mid-2025 — treat any “N requests/month” framing of Cursor pricing as obsolete, not merely dated.

Architecture: Cursor extends VS Code with AI capabilities at multiple layers: inline completions, a chat sidebar, and Agent mode. It maintains a shadow workspace where it can test proposed changes before applying them. Cursor 3 supports up to 8 parallel isolated agents running on separate git branches, plus a cloud-sandboxed Background Agent that converts GitHub issues or Slack messages into draft PRs.

LLM models available: Cursor Pro gives access to current-generation Claude, GPT, and Gemini models, alongside Cursor’s own in-house model, Composer (Composer 2.5 as of May 2026). Users can also configure their own API key for direct provider access.

Terminology note: “Composer” and “Agent mode” are two different things, not interchangeable. Composer is the name of Cursor’s own in-house model — Anysphere trains and ships it as one of the model options in the picker. Agent mode is the autonomous multi-file execution feature: it accepts a task, proposes a plan, and executes changes across multiple files, and can run terminal commands in Yolo mode (automatic execution without prompting). Agent mode can be pointed at Composer or at any of the other bundled models.

Context mechanisms:

- **@codebase** — full codebase semantic search

- `@docs` — pull in documentation from URLs
- `@web` — web search integration
- `@file`, `@folder` — explicit file/folder references
- `@git` — reference git history and diffs

Rules: Cursor supports `.cursorrules` files at the project root (and global rules) that provide persistent instructions to the AI — similar to Claude Code’s `CLAUDE.md` pattern.

Provider flexibility: Limited. While Cursor allows “BYO API key” (bring your own Anthropic/OpenAI key), it is not natively designed for OpenRouter or arbitrary providers, and it has no local LLM support. The product is optimised for the models it bundles.

Strengths: - Polished, deeply integrated AI coding experience - Agent mode is very capable, and now supports parallel multi-branch agents - Semantic codebase indexing enables large-codebase context - Large community, extensive documentation, many tutorials - Tab completion is best-in-class

Weaknesses: - IDE-bound — not a CLI tool - Limited model flexibility — still not OpenRouter compatible, no local model support - Proprietary — you are dependent on Anywhere remaining a going concern and maintaining pricing - Heavier than a terminal tool — requires a full VS Code instance - Usage-based credit pricing makes monthly cost less predictable than the old flat-request model

5.2 Windsurf (now Devin Desktop, Cognition AI)

What it is: Windsurf was an AI-first IDE built by Codeium, positioned as a direct competitor to Cursor. Its standout feature was the Cascade agent, which used “Flow” — a system that maintained contextual awareness across an entire coding session rather than treating each interaction as independent. The product has since been acquired and rebranded; treat “Windsurf” as the legacy name and “Devin Desktop” as the current one.

Ownership timeline: Cognition AI (maker of Devin) acquired Windsurf/Codeium in **December 2025** for roughly \$250M, then rebranded the product to **Devin Desktop** on **June 2, 2026** via an over-the-air update — existing settings and plans carried over automatically.

Cascade, the former Windsurf agent, reached end-of-life on **July 1, 2026**, replaced by **Devin Local**: a Rust-based rewrite that Cognition claims is roughly 30% more token-efficient and adds subagent support. Readers searching for either “Windsurf” or “Devin Desktop” should land on this section.

Company: Cognition AI.

Pricing: Devin Desktop has its own pricing ladder, separate from Cognition’s cloud Devin product (see the Devin entry under Cloud Platform Agents — do not conflate the two):

- **Free** — limited quota
- **Pro** — \$20/month
- **Max** — \$200/month
- **Teams** — \$80/month base + \$40/seat

Architecture: Devin Desktop remains a fork of VS Code (as Windsurf was). Devin Local is now its agentic component, having fully superseded Cascade as of July 1, 2026.

LLM models: Devin Desktop uses Cognition-managed inference by default, with some degree of model selection in higher tiers.

Provider flexibility: Local LLM support is now partial — Devin Local can run against local models, while the cloud-backed Devin Cloud path remains Cognition-managed inference only. Not OpenRouter-compatible.

Strengths: - Devin Local carries forward Cascade’s session-awareness strengths with better token efficiency - Historically cheaper tiers than Cursor at the low end - Autocomplete remains strong (inherited from Codeium’s original product) - Clean, polished IDE experience

Weaknesses: - IDE-bound - Two ownership changes and a rebrand in under a year — evaluate stability before committing long-term - Less model flexibility than Cursor - Smaller community than Cursor; documentation is still catching up to the rebrand - Easy to confuse with Cognition’s separate cloud Devin product, which has different pricing and a different execution model

5.3 Zed AI

What it is: Zed is a high-performance code editor written in Rust, designed for speed and minimal latency. Its AI integration (Zed AI) brings Claude as the default model into this fast editor experience.

Architecture: Zed is not a VS Code fork — it is a ground-up implementation in Rust with native GPU acceleration. This makes it significantly faster than Electron-based editors (VS Code, Cursor, Devin Desktop). AI features are integrated via an assistant panel. Zed’s **Agent Client Protocol** (ACP, launched January 2026) is a distinct interoperability feature none of the other dedicated AI IDEs offer: it lets Claude Code, Codex CLI, Gemini CLI, and OpenCode run directly inside Zed as pluggable agents, rather than requiring Zed’s own agent to do everything.

LLM models: Claude is the default. The supported provider list is now extensive: OpenAI, Gemini, Amazon Bedrock, DeepSeek, GitHub Copilot, LM Studio, Mistral, Ollama, OpenRouter, and Vercel.

Pricing: Zed the editor is free and open source. Zed AI is billed separately on token-based usage, restructured into three tiers:

- **Personal** — free permanently — 2,000 accepted edit predictions/month
- **Pro** — \$10/month — unlimited edit predictions, \$5 of tokens included, overage billed at API list price + 10%
- **Business** — \$30/seat/month

Strengths: - Extraordinarily fast editor — the best performance of any AI-integrated editor - Open source editor core - Genuinely broad provider flexibility, including local (Ollama) and OpenRouter — no longer just “supports configuring alternative backends,” both are confirmed Yes - ACP lets you bring your preferred CLI agent (Claude Code, Codex CLI, etc.) into the editor instead of relying solely on Zed’s own agent

Weaknesses: - Smaller plugin/extension ecosystem (incompatible with VS Code extensions) - Native agentic mode is still less mature than Cursor’s or Devin Desktop’s - Primarily macOS and Linux (Windows support in progress) - Token-based overage billing can be less predictable than a flat monthly fee for heavy users

5.4 Google Antigravity 2.0

What it is: Antigravity is Google’s agent-first IDE, unveiled at I/O 2026. Unlike Cursor or Zed, which retrofit agent capability onto a code-editing surface, Antigravity is built around two primary views: an Editor View for direct code work, and an async **Manager Surface** for orchestrating and supervising multiple agents working in parallel across tasks.

Company: Google (Cloud/Developer Products, the same organisation behind Gemini CLI — not Google DeepMind).

Pricing: Free for individuals.

LLM models: Model-agnostic — ships with Gemini 3 Pro, Claude Sonnet 4.5, and GPT-OSS available out of the box.

Why it matters here: Google deprecated the consumer-tier free access to Gemini CLI on June 18, 2026 (Gemini Code Assist for Individuals, Google AI Pro, and Google AI Ultra all stopped serving requests to individual users — see the Gemini CLI entry). Antigravity is Google’s designated landing place for those displaced consumer users, and it is free where

Gemini CLI’s consumer tier no longer is. Anyone evaluating Gemini CLI purely for its old zero-cost tier should look here first.

Confidence note: This section is sourced from a single independent research pass, not cross-verified by a second source. The existence of the product, its free-for-individuals pricing, and its model-agnostic support for Gemini/Claude/GPT-OSS are reasonably solid; treat any finer-grained feature claims (beyond the Editor View / Manager Surface split) as provisional pending a future refresh.

Strengths: - Free for individual use - Model-agnostic rather than locked to Google’s own models - Manager Surface is a genuinely different multi-agent orchestration paradigm from the single-agent-per-session model most competitors use

Weaknesses: - Newest entrant in this table — least battle-tested, thinnest independent documentation - Feature depth outside the two core views is not yet well-verified - Provider/local-model support beyond the three bundled models is unconfirmed

6 Cloud and Web Platform Agents

6.1 OpenHands (formerly OpenDevin)

What it is: OpenHands is a highly capable open-source AI software engineer designed to solve complex software engineering tasks autonomously. It is the open-source project that most directly competes with commercial cloud agents like Devin.

Author and licence: MIT licence. github.com/OpenHands/OpenHands — the project migrated from the All-Hands-AI org to OpenHands; the new path is now the canonical one.

Community size: Approximately 79,600 GitHub stars, 7,000+ commits (July 2026). One of the largest open-source AI coding projects by community size.

Architecture: OpenHands runs in a Docker container with a sandboxed execution environment. This is the key architectural difference from CLI tools — it doesn’t run directly on your machine; it runs in an isolated environment with full access to the shell, filesystem, and browser within that sandbox. This makes it safer for autonomous long-running tasks but adds Docker as a dependency. The project has recently restructured its core: the agent code now lives in separate repos (**software-agent-sdk**, **agent-canvas**), and the primary interface is the browser-based **Agent Canvas** rather than the older “Web UI” branding.

Interfaces:

- **Agent Canvas** — the primary interface, a browser-based UI for interacting with the agent
- **CLI** — a lightweight binary available at github.com/OpenHands/OpenHands-CLI
- **API** — programmable access for CI/CD integration

Installation:

Via Docker (recommended)

```
docker pull docker.all-hands.dev/all-hands-ai/runtime:0.39-nikolaik
```

Lightweight CLI binary (for terminal-first use)

```
pip install openhands-cli
```

LLM provider support: OpenHands supports multiple LLM providers via litellm, including Anthropic, OpenAI, Google, and others.

MCP support: OpenHands supports MCP server configuration across three transport types — SSE, Streamable HTTP, and stdio:

```
{
  "mcpServers": {
    "fetch": {
      "command": "uvx",
      "args": ["mcp-server-fetch"]
    },
    "notion": {
      "url": "https://mcp.notion.com/mcp"
    }
  }
}
```

For stdio servers, a proxy approach (SuperGateway/FastMCP) is recommended over direct stdio for reliability. Note: the claim that OpenHands “natively discovers MCP tools automatically” remains refuted — MCP is configured explicitly, not auto-discovered.

Strengths: - Very capable at complex, long-horizon tasks — closest open-source equivalent to Devin - Docker sandboxing means it can’t accidentally break your host system - Full MCP support across three transport types - Massive community and active development - Agent Canvas is more accessible than CLI tools

Weaknesses: - Requires Docker — heavier than CLI tools - Primarily a cloud/web platform — less suitable for quick interactive sessions - Slower to start up than CLI tools (Docker container spin-up) - Not a terminal-first tool despite having a CLI option

6.2 Devin (by Cognition AI)

What it is: Devin was the first high-profile “AI software engineer” capable of autonomously solving complex GitHub issues end-to-end. It is a commercial cloud product from Cognition AI.

Pricing: Core plan is \$20/month base plus pay-as-you-go ACU billing at \$2.25/ACU (1 ACU 15 minutes of autonomous work, roughly \$9/hour of Devin work). Team plan is \$500/month with 250 ACUs included, \$2.00/ACU overage; Enterprise is custom (SSO, VPC). Realistic all-in cost for an individual with moderate use runs **\$70–220/month** — a meaningfully more expensive picture than a flat “\$20/month” headline suggests, though far below the old “thousands per month” reputation. Whether that’s viable depends on usage: light, occasional use can land near the bottom of the range, but sustained daily use pushes toward the top of it and past what most individual developers pay for Claude Code or a CLI tool.

Devin now sits alongside a sibling product, **Devin Desktop** (formerly Windsurf, rebranded June 2026 after Cognition’s acquisition of Windsurf) — see the Windsurf/Devin Desktop entry for that product’s separate pricing ladder. The two are distinct: Devin (this entry) is the cloud autonomous-agent product with ACU billing; Devin Desktop is an IDE with its own subscription tiers.

Strengths: - Very capable for long-horizon autonomous tasks - Well-integrated with GitHub workflows - No longer prohibitively priced for individuals at light-to-moderate usage levels

Weaknesses: - ACU-based billing makes costs harder to predict than a flat subscription; heavy use is still expensive relative to CLI-based alternatives - Fully proprietary — no control over the stack - Not a CLI tool

7 Commercial AI Coding Assistants

7.1 GitHub Copilot with Agent Mode

What it is: GitHub Copilot is Microsoft/GitHub’s AI coding assistant. It now spans autocomplete, chat, an in-IDE “Agent Mode,” and a separate cloud-based “Copilot Coding Agent.” These are two distinct products, not one: **Agent Mode** (VS Code, synchronous, runs locally, GA since early 2026) executes edits directly in your editor session; the **Copilot Coding Agent** (cloud, asynchronous, GA September 2025) takes a GitHub issue assignment and opens a pull request without an open editor session. Both superseded the original “Copilot Workspace” preview, which was sunset May 30, 2025 — that name no longer refers to a current product.

Pricing (restructured with usage-based billing, June 1, 2026):

- **Free** — \$0/month; 2,000 completions/month
- **Pro** — \$10/month; unlimited completions + \$15/month AI credits, \$0.04/premium request after quota

- **Pro+** — \$39/month; unlimited completions + \$70/month AI credits; premium model access including Claude Opus
- **Individual Max** — \$100/month; unlimited usage + \$200/month AI credits
- **Business** — \$19/user/month
- **Enterprise** — \$39/user/month

LLM provider: Copilot was never as OpenAI-locked as it appeared. The built-in model picker (no key required) already includes Claude Opus 4.5/4.6, Claude Sonnet 4.5/4.6, and Kimi K2.7 Code alongside GPT-5.x-Codex variants. More significantly, **BYOK (Bring Your Own Key) is now generally available** for Business/Enterprise customers (VS Code, April 22, 2026; Copilot desktop app, June 23, 2026), supporting Anthropic, Gemini, OpenRouter, Ollama, Azure OpenAI, Foundry Local, and any OpenAI-compatible endpoint. BYOK covers chat and agent workflows only — not code completions. Individual Free/Pro/Pro+ users get provider flexibility through VS Code’s Language Model Chat Provider API instead of full BYOK.

OpenRouter support: Now Yes for Business/Enterprise via BYOK, and available to individual users through the Language Model Chat Provider API. The earlier “not supported” verdict is outdated.

Strengths: - Deep GitHub integration — issues, PRs, code review - Real provider flexibility now, not just a wide built-in model picker - \$10/month remains the most affordable commercial completions tier - Enterprise features and security compliance

Weaknesses: - BYOK is gated to Business/Enterprise — individual-tier flexibility is narrower and completion-only workflows still don’t benefit from BYOK at all - Agent Mode and Copilot Coding Agent are two separate mental models to learn, with different sync/async execution semantics - Less suited for non-GitHub workflows

7.2 Amazon Q Developer (formerly CodeWhisperer) — being sunset, see **Kiro**

What it is: Amazon Q Developer is AWS’s AI coding assistant, with deep integration into the AWS ecosystem (rebranded from CodeWhisperer in 2024). Its agentic mode autonomously implements features, refactors, and proposes multi-step changes in both IDE and CLI — this is no longer a completions-only tool, contrary to earlier characterisations.

CLI agentic mode: Supports Claude Sonnet 4 / 3.7 / 3.5 via `/model`, with file read/write, bash execution, and multi-step autonomy — a genuine general-purpose coding agent, not an AWS-console-only assistant.

Pricing:

- **Free** — 50 agentic requests/month
- **Pro** — \$19/user/month; 10,000 inference calls (~1,000 user requests)

Important — deprecation in progress: AWS is sunseting Q Developer’s IDE plugins and paid subscriptions. New signups closed May 15, 2026; end of support is April 30, 2027. **AWS Kiro is the designated successor** (see below) — do not recommend Q Developer to a new adopter without flagging this.

Strengths: - Best-in-class for AWS-heavy workflows - Security vulnerability scanning, IAM policy generation - CLI agentic mode is now genuinely general-purpose, not AWS-console-limited

Weaknesses: - Being wound down — new signups already closed - AWS-focused — less useful for non-AWS codebases - Provider choice limited to Claude via Bedrock/CLI; no OpenRouter or broad model-agnosticism

7.3 JetBrains AI Assistant

What it is: JetBrains AI Assistant is integrated into IntelliJ IDEA, PyCharm, WebStorm, GoLand, and all other JetBrains IDEs. It provides chat, completion, and an autonomous multi-step agent called **Junie** — worth naming explicitly, since it is JetBrains’ direct answer to agentic coding assistants rather than a generic “agentic features” bundle.

Pricing (fully decoupled from the IDE subscription):

- **AI Free** — \$0
- **AI Pro** — Individual \$10/month / Business \$20/month
- **AI Ultimate** — Individual \$30/month / Business \$60/month
- **AI Enterprise** — custom

The old “~\$24.90/month All Products Pack” bundling is gone; AI features are now priced and sold independently of the IDE licence.

LLM models: Model rotation includes Claude 4.0 Sonnet, Claude 4.5 Haiku, Gemini 2.5/3.0, Azure GPT-5, and Qwen 2.5. Local model support (Ollama, LM Studio, llama.cpp) is confirmed on IDEs v2025.1 and later.

Strengths: - Best choice if you are already heavily invested in JetBrains IDEs - Deep IDE integration (refactoring, inspections, test generation) - Junie provides genuine autonomous multi-step agent capability, not just chat - Local model support now confirmed, closing a prior gap

Weaknesses: - Junie is less proven than Cursor’s agent mode or Cline on complex multi-file tasks - Pricing is now a separate subscription on top of (or instead of) the IDE licence, adding cost-tracking overhead for existing JetBrains customers - JetBrains IDEs remain heavier than VS Code

7.4 Tabnine

What it is: Tabnine is an AI coding assistant with a long history (one of the first serious AI coding tools), historically focused on fast, high-quality inline completions. It has since pivoted toward agentic workflows.

Pricing (materially higher than before — the old ~\$12/month Pro tier was sunset in 2025):

- **Code Assistant** — \$39/user/month
- **Agentic Platform** — \$59/user/month; adds MCP integration, autonomous workflows, CLI access
- **Enterprise** — custom, with self-hosted model option

LLM provider: Tabnine uses its own models plus third-party models (Anthropic, etc.). Enterprise tier still supports self-hosted deployment for data sovereignty.

Strengths: - Very fast completions (latency-optimised) - Enterprise self-hosted option — best for organisations with strict data policies - Agentic Platform tier now offers MCP integration and CLI access, closing the gap with agent-first tools

Weaknesses: - Pricing jumped sharply (\$12 → \$39/\$59/month) — no longer the budget completions option it once was - Agentic capability is newer and less proven than Cursor, Cline, or Claude Code - Being overtaken by Cursor/Windsurf on raw completion quality

7.5 Supermaven — discontinued, folded into Cursor

What it is: Supermaven was a VS Code-focused autocomplete tool with a very large context window for completions (300k tokens), notable for speed and accuracy.

Status: Supermaven is no longer an independent product. It was **acquired by Anysphere (Cursor’s parent company) in November 2024**, not by Cursor itself as a product — Anysphere absorbed the team and technology, and Cursor has integrated Supermaven’s completion engine into its tab-completion feature. If Supermaven was your tool of choice, Cursor is its direct successor; there is no standalone Supermaven product to evaluate separately.

8 Cross-Cutting Analysis

8.1 Provider Flexibility Analysis

A critical factor for Claude Code users concerned about API costs is whether an alternative tool is model-agnostic — allowing you to route requests to cheaper models, local models, or whichever provider has the best current pricing.

8.1.1 OpenRouter Compatibility

OpenRouter is a unified API gateway that provides access to hundreds of models from dozens of providers under a single API key, with pay-per-token pricing and no monthly subscription.

Tool	OpenRouter Support	Notes
Aider	Yes (verified)	Full setup at aider.chat/docs/llms/openrouter.html via litellm.
Cline	Yes (verified)	Listed on openrouter.ai/works-with-openrouter .
OpenCode	Yes (verified)	75+ providers incl. OpenRouter; Claude itself now needs a raw API key since the Jan 2026 split (OpenRouter routing unaffected).
Continue.dev	Yes	Via OpenRouter provider in <code>config.yaml</code> .
Goose	Partial	Reachable via custom API endpoint config; not an explicitly named provider in Goose's own docs.

Tool	OpenRouter Support	Notes
Amp (Sourcegraph)	Partial	Model-agnostic, pay-as-you-go; OpenRouter not explicitly documented as a named provider.
OpenAI Codex CLI	Partial	ChatGPT subscription is the native path; local via <code>--oss</code> ; OpenRouter reachable via API-compatible config, not a first-class integration.
Plandex	Partial	Via OpenAI- compatible endpoint.
SWE-agent	Partial	Typically direct Claude/OpenAI; OpenRouter- capable but not the primary pathway.
OpenHands	Partial	Via litellm provider abstraction; not a first-class named provider in docs surfaced this round.

Tool	OpenRouter Support	Notes
GitHub Copilot	Yes (was No)	Via BYOK, GA for Business/Enterprise (April 2026 VS Code, June 2026 desktop app); individual tier via VS Code Language Model Chat Provider API. Does not cover code completions.
GitHub Copilot CLI	Partial*	Model picker spans Claude/GPT/Gemini; explicit OpenRouter support unconfirmed.
Zed AI	Yes (was Partial)	Explicitly listed alongside 10+ other providers (Bedrock, DeepSeek, Copilot, LM Studio, Mistral, Ollama, Vercel).

Tool	OpenRouter Support	Notes
Google Antigravity 2.0	Partial*	Described as “model-agnostic” (Gemini 3 Pro, Claude Sonnet 4.5, GPT-OSS); OpenRouter specifically unconfirmed.
Cursor	No	BYO key for direct providers only.
Windsurf (→ Devin Desktop)	No	Cognition- managed inference or Devin Local; not OpenRouter- oriented.
Devin	No	Cognition- managed inference only (ACU billing).
Gemini CLI	No	Locked to Gemini models.
Amazon Q Developer	No	Locked to Ama- zon/Anthropic via Bedrock.
AWS Kiro IDE	No	AWS-native model routing.
JetBrains AI Assistant	No	Tied to JetBrains’ own model rotation, not user-configurable OpenRouter.

Tabnine	No	Own models plus a fixed provider list.
---------	----	--

* Unverified — not independently confirmed by this refresh’s sourced research; treat as an open question rather than a confirmed capability.

Conclusion: For OpenRouter flexibility, the open-source CLI tools (Aider, Cline, OpenCode, Continue.dev, Goose, Amp) remain your best options, and the pool of flexible tools has widened in 2026 — Zed AI now confirms OpenRouter as one of its ten-plus provider integrations, and OpenAI’s own Codex CLI supports local/OSS models via its `--oss` flag even though it isn’t OpenRouter-routed. The old blanket claim that commercial IDE tools are locked to their own ecosystems no longer holds without qualification: GitHub Copilot rolled out Bring-Your-Own-Key (BYOK) as GA in 2026 — first for VS Code Business/Enterprise (April 22, 2026), then the Copilot desktop app (June 23, 2026) — covering Anthropic, Gemini, OpenRouter, Ollama, Azure OpenAI, Foundry Local, and any OpenAI-compatible endpoint. This is a real exception, not a marginal one, given Copilot was previously this document’s exemplar of zero provider flexibility. Two caveats keep it from being a full reversal: BYOK applies to chat/agent workflows only (not code completions), and it is gated to Business/Enterprise plans — individual Free/Pro/Pro+ users get flexibility only through VS Code’s Language Model Chat Provider API, a narrower mechanism. Cursor and Windsurf (now rebranded Devin Desktop under Cognition AI) remain the clearest holdouts: both are still closed to OpenRouter and restrict BYO keys to direct providers only.

8.1.2 Local LLM Support (Ollama / LM Studio)

Running models locally eliminates API costs entirely. Quality of locally-runnable models has continued to improve — Alibaba’s Qwen3.5-9B has become a mainstream local choice for coding and RAG workloads (viable on 16GB VRAM), and the sub-35B Qwen3 model family remains Apache 2.0-licensed for unrestricted commercial use and fine-tuning.

Tool	Ollama	LM Studio	Notes
Aider	Yes	Yes	Via litellm.
Cline	Yes	Yes	Native provider options.
OpenCode	Yes	Yes	Provider-agnostic architecture.

Tool	Ollama	LM Studio	Notes
Continue.dev	Yes	Yes	localhost:11434 default.
Goose	Yes	Partial	On-device philosophy is core; MCP-configurable local providers.
Amp (Sourcegraph)	Yes	Partial	CLI-local execution is one of its two hybrid modes (vs. cloud “Orbs”).
OpenAI Codex CLI	Yes	Yes	Via <code>--oss</code> flag; also supports MLX.
Plandex	Partial	Partial	Via OpenAI-compatible API config.
OpenHands	Yes	Yes	Via <code>litellm</code> ; Agent Canvas can be self-hosted.
GitHub Copilot	Yes (was No)	Yes	Via BYOK (Ollama, Microsoft Foundry Local), Business/Enterprise, GA April 2026.
GitHub Copilot CLI	Partial*	Partial*	Not confirmed by sourced research this round.
JetBrains AI Assistant	Yes (was not listed)	Yes	Ollama, LM Studio, llama.cpp confirmed on IDEs v2025.1+.
Zed AI	Yes	Partial	Ollama explicitly supported.

Tool	Ollama	LM Studio	Notes
Windsurf (→ Devin Desktop)	Partial (was No)	No	Devin Local variant now available (Rust, ~30% more token-efficient); Devin Cloud remains cloud-only.
Devin	Partial	No	Same Devin Local note — Devin Cloud itself is not locally runnable.
Cursor	No	No	Cloud models only.
Gemini CLI	No	No	Locked to Gemini.
Amazon Q Developer	No	No	AWS/Bedrock-locked.
AWS Kiro IDE	No	No	AWS-native.
Tabnine	No	No	Cloud/on-prem/air-gapped deployment options exist, but not consumer local-model tools like Ollama.
Google Antigravity 2.0	Unverified*	Unverified*	No sourced information either way.

* Unverified — not independently confirmed by this refresh’s sourced research; treat as an open question rather than a confirmed capability.

Best tools for local LLMs: Aider, Cline, OpenCode, Continue.dev, and Goose remain first-class options for local model use, and the field has broadened. JetBrains AI Assistant now confirms local model support (Ollama, LM Studio, llama.cpp) on IDEs v2025.1+, a capability the

previous edition of this document didn't credit it with. Amp and OpenAI's Codex CLI (via `--oss`, supporting Ollama, LM Studio, and MLX) both ship local-model paths as first-class options rather than afterthoughts. Cognition's rebranded Windsurf-turned-Devin-Desktop is a partial case: Devin Local (its Rust-based, on-device agent, roughly 30% more token-efficient than its predecessor) supports local inference, but Devin Cloud remains Cognition-managed only — don't conflate the two when evaluating this product.

Recommended local models for coding (as of mid-2026):

- **Qwen3.5-9B** — mainstream local choice for coding and RAG workloads, viable on 16GB VRAM
- **Qwen3-Coder** family (Next / Plus / 480B-A35B variants) — sub-35B variants are Apache 2.0-licensed for commercial use and fine-tuning
- **DeepSeek V4** (flash/pro) — strong reasoning-capable option; note the older `deepseek-chat/deepseek-reasoner` API models are sunset July 24, 2026, so local deployments should track the V4 line
- Hardware requirement guidance is largely unchanged: 16-24GB VRAM covers most serious 9B-32B-class local coding models; larger MoE variants need considerably more.

8.2 Feature Comparison Matrix

The matrix spans 16 tools, too many to render legibly as one table at this page width — split below into four groups of four.

Group 1: Claude Code, Aider, OpenCode, Cline

Feature	Claude Code	Aider	OpenCode	Cline
Interface	CLI	CLI	TUI	VS Code / CLI
Agentic file editing	Yes (full)	Yes (full)	Yes (full)	Yes (full)
Multi-file context	Yes	Yes	Yes	Yes
Shell execution	Yes	Yes	Yes	Yes
Web search	Yes	No	Partial	Yes
Browser use	Via MCP/WebFetch†	No	No	Yes
MCP support	Yes (native)	No	Yes	Yes

Feature	Claude Code	Aider	OpenCode	Cline
Custom hooks	Yes (skills+hooks)	No	Limited	.clinerules
OpenRouter	No	Yes	Yes	Yes
Local LLMs	No	Yes	Yes	Yes
Open source	No	Yes (MIT)	Yes (MIT)	Yes (MIT)
Git integration	Yes	Yes (auto-commit)	Yes	Yes
Cost model	Max plan or API	API only	API only	API only
Self-hosted option	No	N/A	N/A	N/A

Group 2: Continue.dev, Goose, Gemini CLI, OpenHands

Feature	Continue.dev	Goose	Gemini CLI	OpenHands
Interface	VS Code / JetBrains	CLI + Desktop	CLI	Web / CLI
Agentic file editing	Yes (partial)	Yes (full)	Yes (full)	Yes (full)
Multi-file context	Yes	Yes	Yes (1M tokens) [†]	Yes
Shell execution	Yes (agent mode)	Yes	Yes	Yes (sandboxed)
Web search	No	Yes	Yes	Yes
Browser use	No	No	No	Yes
MCP support	Partial	Yes (native) [§]	Yes	Yes (config)
Custom hooks	config.yaml	Extensions	Settings	Limited
OpenRouter	Yes	Partial	No	Partial
Local LLMs	Yes	Yes	No	Yes
Open source	Yes (Apache-2)	Yes (Apache-2)	Yes (Apache-2)	Yes (MIT)
Git integration	Yes	Yes	Yes	Yes
Cost model	API only	API only	Enterprise only [¶]	API only

Feature	Continue.dev	Goose	Gemini CLI	OpenHands
Self-hosted option	N/A	N/A	N/A	Yes

Group 3: Cursor, Windsurf, Codex CLI, Amp

Feature	Cursor	Windsurf	Codex CLI	Amp
Interface	IDE	IDE (now Devin Desktop)	CLI	CLI + Cloud
Agentic file editing	Yes (full)	Yes (full)	Yes (full)	Yes (full)
Multi-file context	Yes	Yes	Yes	Yes (code-graph)
Shell execution	Yes (yolo mode)	Yes	Yes	Yes
Web search	Yes	Yes	No	Unverified
Browser use	No	No	No	No
MCP support	No	No	Unverified	Unverified
Custom hooks	.cursorrules	Limited	Limited	Yes (event hooks)
OpenRouter	No	No	Partial	Partial
Local LLMs	No	Partial	Yes	Yes
Open source	No	No	Yes	No
Git integration	Yes	Yes	Yes	Yes
Cost model	\$20-200/mo + API	Free / \$20 / \$200 / Teams	Bundled ChatGPT	Pay-as-you-go, no markup
Self-hosted option	No	No	No	No

Group 4: Devin, Copilot, Kiro, Antigravity

Feature	Devin	Copilot	Kiro	Antigravity
Interface	Cloud / Web UI	IDE + Cloud	IDE + CLI + Web	IDE (Editor + Manager)
Agentic file editing	Yes (full, sandboxed)	Yes (full)	Yes (full)	Yes (full)*
Multi-file context	Yes	Yes (Enterprise)	Yes	Yes*
Shell execution	Yes (sandboxed)	Yes	Yes	Unverified
Web search	Unverified	Yes	Yes	Unverified
Browser use	Yes	No	No	Unverified
MCP support	Unverified	Yes	Yes (full)	Unverified
Custom hooks	Limited	Limited	Yes (steering rules)	Unverified
OpenRouter	No	Yes	No	Partial*
Local LLMs	Partial	Yes	No	Unverified
Open source	No	No	No	No
Git integration	Yes	Yes	Yes	Unverified
Cost model	\$20/mo + \$2.25/ACU	\$10-100/mo indiv.	\$20-200/mo (credits)	Free (individuals)
Self-hosted option	No	No	No	Unverified

9 * Unverified/lower-confidence — see the relevant tool’s section for sourcing caveats. † Gemini CLI’s 1M-token context window is a model-level feature, independent of the consumer free tier’s June 2026 deprecation (see Cost Analysis). ‡ Corrected from a bare “Yes”: Claude Code’s browser access is via MCP servers (e.g. Playwright) or WebFetch, not a built-in browser like Cline’s Puppeteer integration. § Corrected from a hedged claim — Goose’s MCP support is a confirmed, core architectural pillar (70+ extensions, 3,000+ compatible servers), not a disputed one. ¶ Gemini CLI’s consumer free tier ended June 18, 2026; only enterprise Code Assist tiers remain.

9.1 Extensibility Deep Dive

One of Claude Code’s most powerful features is its extensibility system: skills (reusable prompt templates invoked as `/commands`), hooks (shell commands that fire on events like tool calls or session start), and MCP servers (external tools exposed via a standard protocol). This combination allows you to build a highly personalised, automated coding workflow. How do the alternatives compare?

9.1.1 Claude Code’s Extension Architecture

For context, Claude Code’s three-layer system works as follows:

1. **Skills** — Markdown files stored in `~/.claude/skills/` that define reusable workflows. When invoked as `/skill-name`, their content is loaded and executed. Skills can include checklists, decision trees, and instructions for specific types of tasks (debugging, code review, PR creation).
2. **Hooks** — Shell commands configured in `settings.json` that fire on specific Claude Code events. For example: run linting before a commit, post a notification when a session ends, validate that tests pass after edits. Hooks are executed by the Claude Code harness, not by the AI — this makes them reliable and deterministic.
3. **MCP Servers** — External processes exposing tools via the Model Context Protocol.

Claude Code can call these tools as part of its reasoning (read from a database, query an external API, access browser dev tools, etc.). Any MCP server in the ecosystem works.

9.1.2 How Alternatives Compare

Aider: No equivalent to skills or hooks. No MCP support. Configuration via `.aider.conf.yml` covers model settings and preferences but not custom workflows. *Least extensible of the serious alternatives.*

OpenCode: MCP support confirmed. Some configuration of workflows via the config system. No equivalent to Claude Code’s hooks system or skills paradigm. *Mid-range extensibility.*

Cline: MCP support confirmed. `.clinerules` files provide persistent instructions analogous to `CLAUDE.md`. No hook system. The Plan/Act split provides some workflow control but it’s not programmable. *Better than average but no hooks.*

Continue.dev: The “blocks” system provides a plugin-like architecture for adding new context providers and tools. Config-driven customisation is deep. No hook system. MCP support via tool configuration. *Good for IDE users, weaker hook/automation story.*

Goose: MCP is a core architectural pillar, not a bolt-on — Goose’s extension system is built directly on MCP, with 70+ documented extensions and 3,000+ compatible MCP servers. It’s the closest conceptual match to Claude Code’s MCP+skills combination, though the format differs, and it has no hook system equivalent. Goose is also a co-steward, alongside Anthropic’s MCP and OpenAI’s AGENTS.md, of the Linux Foundation’s Agentic AI Foundation (est. December 2025) — see the AGENTS.md note below. *Strong extensibility story, and a governance role to match.*

Gemini CLI: MCP support confirmed. Extensions mechanism for adding capabilities. No hook system. *Good starting point for building a custom workflow*, though note its consumer free tier ended June 18, 2026 (see Cost Analysis).

OpenHands: MCP support confirmed, with three transport types (SSE, Streamable HTTP, stdio) — a proxy approach (SuperGateway/FastMCP) is recommended over direct stdio for reliability. Configuration is required; MCP tools are not auto-discovered. Limited hook/automation equivalent otherwise. *Weakest extensibility story among the actively-developed alternatives.*

9.1.3 The AGENTS.md Open Standard

Alongside MCP, a second cross-cutting convention has emerged that matters for anyone comparing extensibility stories: **AGENTS.md**, an open standard for project-level agent instructions. It was formalized in August 2025 by a coalition including OpenAI, Google, Cursor, and Factory, and donated to the Linux Foundation’s Agentic AI Foundation in December 2025 — the same body Goose joined as a co-steward (see above). It is now read by 30+ agents and tools, including Codex, Copilot, Cursor, Gemini CLI, Jules, Aider, Zed, Windsurf, and Devin. Claude Code itself supports it, via import into **CLAUDE.md**.

The distinction from MCP matters: MCP standardises *tool/capability* portability (how an agent reaches external systems); AGENTS.md standardises *project-instruction* portability (how an agent learns a repo’s conventions, build steps, and constraints). They’re complementary, not competing — a tool can support one, both, or neither. For anyone currently maintaining a pile of per-tool rule files (`.clinerules`, `.cursorrules`, **CLAUDE.md**), AGENTS.md is worth tracking as a potential single source of truth that many tools can read natively, reducing the duplication of instructions across tools.

9.1.4 What This Means for Migration

If you have heavily invested in Claude Code’s extensibility system (skills, hooks, MCPs), a complete migration requires rebuilding your workflow on whatever alternative you choose. The good news: MCP server investments are portable to any MCP-compatible tool (Cline, OpenCode, Gemini CLI, Goose, OpenHands, Amp). The skills system has no direct equivalent in any alternative — the closest analogues are `.clinerules` (Cline), **CLAUDE.md**-style instruction files, or custom system prompts. The hooks system is the hardest to replicate — only Claude Code has deterministic shell-command hooks that fire on tool events. Alternatives would require wrapping the tool in a shell script or using git hooks as a partial substitute. If your per-tool instruction files (rather than MCP servers) are the bigger migration cost, look at consolidating onto AGENTS.md — it’s read natively by a wider set of tools than any single vendor’s rule-file format, and Claude Code can import it into **CLAUDE.md**.

9.2 Cost Analysis

If you are a heavy Claude Code user currently on a Max plan, what would the same usage cost on each alternative? Note first that “unlimited usage” is not an accurate description of the Max plan: Max 5x (\$100/month) and Max 20x (\$200/month) both carry a 5-hour rolling session cap (~225 messages/window on Max 5x, ~900 on Max 20x) *and* a separate weekly cap — one ceiling across all models, another specifically for Sonnet models. The session limit was

doubled and the peak-hour reduction removed on May 6, 2026, which improved things, but genuinely unbounded usage was never part of the plan.

9.2.1 Defining “Heavy Usage”

For this analysis, heavy usage means: - ~4-8 hours of active coding per day - Typical session: long context (50k-200k tokens input), many file edits, running tests - Estimated: ~100-200 million input tokens per month, 10-20 million output tokens per month

These are rough estimates — actual token consumption varies enormously by workflow and model. The worked examples below use the midpoints (~150M input / ~15M output tokens per month).

9.2.2 API Cost Estimates (Direct Provider, as of July 2026)

Pricing changes frequently; treat these as ballpark figures using current published rates, not guarantees.

Anthropic Claude Sonnet 5: - Input: \$2/million tokens (introductory, through Aug 31, 2026; rising to \$3 thereafter) - Output: \$10/million tokens (rising to \$15 after Aug 31, 2026) - Heavy usage estimate: $(150M \times \$2 + 15M \times \$10) / 1M$ **\$300 + \$150 = ~\$450/month** (rising to ~\$675/month once the introductory rate ends in September) - *This is why a Max plan (5-hour and weekly caps notwithstanding) remains good value for genuinely heavy users, even though it isn't the “unlimited” plan the old framing implied.*

Anthropic Claude Haiku 4.5: - Input: \$1/million tokens - Output: \$5/million tokens - Heavy usage estimate: **\$225/month** - *Cheaper and faster than Sonnet 5, though the fastest/cheapest current Claude tier is no longer as dramatically cheap as the old Haiku 3 figures (\$0.25/\$1.25) this document previously cited — Anthropic's tiering has shifted upward across the board.*

OpenAI GPT-5.4 (mid-tier; GPT-4o is no longer current): - Input: \$2.50/million tokens - Output: \$15/million tokens - Heavy usage estimate: $(150M \times \$2.50 + 15M \times \$15) / 1M$ **\$600/month** - *The flagship GPT-5.5 (\$5/\$30 per million) runs roughly double this; the budget GPT-5.4-nano (\$0.20/\$1.25) is a DeepSeek-tier option if quality suffices.*

Google Gemini 3.1 Pro Preview (paid tier; Pro models left the free tier April 1, 2026): - 200k context: \$2.00 input / \$12.00 output per million tokens - >200k context: \$4.00 input / \$18.00 output per million tokens - Heavy usage estimate: **\$480/month** at 200k context, **\$870/month** above it - *Gemini 3.5 Flash (\$1.50/\$9.00) remains free for light usage with reduced daily quotas, but heavy usage as defined here will exceed the free quota*

— *there is no longer a realistic “\$0 at heavy usage” story for Gemini’s API tier. (Separately, the Gemini CLI product’s* consumer free tier was withdrawn entirely on June 18, 2026 — see below.)**

DeepSeek V4 Flash (via OpenRouter; supersedes the deprecated deepseek-chat/deepseek-reasoner sunset July 24, 2026): - Input: \$0.14/million tokens - Output: \$0.28/million tokens - Heavy usage estimate: **~\$25/month** - *Still dramatically cheaper than any frontier model, with a 1M-token context window at no extra charge.*

Qwen3.5-9B (local via Ollama): - Cost: \$0 — no API costs - Hardware requirement: viable on 16GB VRAM (lower bar than the 24GB the previous Qwen2.5-Coder:32B recommendation needed) - *Effectively free if you have the hardware, and the hardware bar has come down.*

9.2.3 Per-Tool Cost Summary

Scenario	Min Monthly	Heavy Usage (Cloud)	Heavy Usage (Local)	Notes
Aider + Claude Sonnet 5	\$0 tool + API	~\$450/mo (intro pricing, \$2/\$10, through Aug 31 2026); ~\$675/mo from Sept 1 2026 (\$3/\$15)	N/A	Was quoted against Claude 3.5 Sonnet (~\$675) in the old doc — coincidentally similar number, different (current) model.
Aider + Claude Haiku 4.5	\$0 tool + API	~\$225/mo	N/A	Cheaper same-vendor fallback; the “reduce cost, stay on Claude” scenario.

Scenario	Min Monthly	Heavy Usage (Cloud)	Heavy Usage (Local)	Notes
Aider + DeepSeek v4-flash (OpenRouter)	\$0 tool + API	~\$25/mo	N/A	Was ~\$27/mo in old doc against deepseek-chat (now deprecated, sunsets 2026-07-24).
Aider + Qwen3-Coder-Next (OpenRouter)	\$0 tool + API	~\$28.5/mo	N/A	Current mainstream cheap-coding-model recommendation, replacing Qwen2.5-Coder.
Aider + Ollama (local)	\$0	\$0	GPU hardware cost only	Recommended local model: Qwen3.5-9B (16GB VRAM viable) rather than Qwen2.5-Coder:32b (24GB+).
OpenCode + DeepSeek v4-flash	\$0 tool + API	~\$25/mo	N/A	OpenCode broke with Anthropic Jan 2026 — Claude now requires a raw API key; doesn't affect this DeepSeek-routed cost.

Scenario	Min Monthly	Heavy Usage (Cloud)	Heavy Usage (Local)	Notes
OpenCode (ChatGPT-native)	\$0 tool + ChatGPT subscription	Bundled — see Codex CLI row (same OpenAI subscription tiers apply since the Jan 2026 OpenAI partnership)	N/A	New row.
Cline + OpenRouter (model varies)	\$0 tool + API	Varies — e.g. ~\$28.5/mo on Qwen3- Coder-Next, ~\$450-675/mo on Claude Sonnet 5	N/A	Route cheaply via OpenRouter.
Amp (Sourcegraph)	Pay-as-you-go, no markup	Not computable — rate card unpublished	Partial (CLI-local mode available)	Sourcegraph has not published per-token rates as of 2026-07-06; see amp- code.com/pricing.
Gemini CLI (individual/consumer)	\$0 (pre-June 18, 2026 only)	N/A — consumer free tier ended June 18, 2026	N/A	If paying API-direct: Gemini 3.1 Flash-Lite ~\$60/mo heavy-usage, Gemini 3.5 Flash ~\$360/mo. See Google Antigravity as the current zero-cost Google option.

Scenario	Min Monthly	Heavy Usage (Cloud)	Heavy Usage (Local)	Notes
Gemini CLI (enterprise, Code Assist Stan- dard/Enterprise)	Custom (commitment- based)	Varies by commitment tier (1,500-2,000 req/day quotas)	N/A	Enterprise is the only tier still served.
Google Antigravity 2.0	\$0 (individuals)	Unverified — no per-token cost data sourced	N/A	Confidence: medium-low (single source). Practical successor to the old “Gemini CLI free tier” pitch.
Cursor Pro	\$20/mo	\$20/mo (usage-based credit pool, not a hard cap; heavy users may need Pro+/Ultra)	N/A	Old “500 fast re- quests/month” framing is gone; credit-pool billing since mid-2025.
Cursor Pro+	\$60/mo	\$60/mo (larger credit pool)	N/A	New tier.
Cursor Ultra	\$200/mo	\$200/mo	N/A	New tier.
Devin Desktop (formerly Windsurf) — Pro	\$20/mo	\$20/mo	Partial (Devin Local)	Replaces “Windsurf Pro ~\$15/month” — both price and product name changed.
Devin Desktop — Max	\$200/mo	\$200/mo	Partial	New tier.
Devin Desktop — Teams	\$80/mo base + \$40/seat	\$80 + \$40/seat	Partial	New tier.

Scenario	Min Monthly	Heavy Usage (Cloud)	Heavy Usage (Local)	Notes
Devin (cloud agent, Core)	\$20/mo base	\$70-220/mo realistic individual total (\$20 base + \$2.25/ACU pay-as-you-go; 1 ACU 15 min autonomous work, ~\$9/hour)	N/A	Replaces both the old “\$500+/mo enterprise-only” claim and a flatter “\$20/month” correction — the real figure is usage-dependent and higher than a flat \$20.
Devin (cloud agent, Team)	\$500/mo (250 ACUs included)	\$500 + \$2.00/ACU overage	N/A	Overage rate now specified.
GitHub Copilot — Individual Free	\$0/mo	\$0 (2,000 completions/mo cap)	N/A	New tier detail.
GitHub Copilot — Individual Pro	\$10/mo	\$10 + \$0.04/premium request after \$15 included AI credits exhausted	N/A	Replaces flat “\$10/month” — now usage-based beyond included credits (billing changed June 1, 2026).
GitHub Copilot — Individual Pro+	\$39/mo	\$39 + overage (\$70 credits included)	N/A	New tier, includes premium models (Claude Opus access).

Scenario	Min Monthly	Heavy Usage (Cloud)	Heavy Usage (Local)	Notes
GitHub Copilot — Individual Max	\$100/mo	\$100 + overage (\$200 credits included)	N/A	New tier.
GitHub Copilot — Business	\$19/user/mo	\$19/user/mo + policy- controlled model catalog	N/A	Unchanged from old doc.
GitHub Copilot — Enterprise	\$39/user/mo	\$39/user/mo	N/A	Unchanged from old doc.
GitHub Copilot CLI	Bundled — inherits Copilot plan tiers above	Same	N/A	No standalone pricing found; open question whether it requires a specific plan tier.
Amazon Q Developer — Free	\$0/mo	\$0 (50 agentic requests/mo cap)	N/A	New row.
Amazon Q Developer — Pro	\$19/user/mo	\$19/user/mo (10,000 inference calls, ~1,000 user requests)	N/A	Also flag the sunset (new signups closed May 15 2026, support ends April 30 2027, Kiro is successor).
AWS Kiro — Free	\$0/mo	\$0 (50 credits)	N/A	New row.
AWS Kiro — Pro	\$20/mo	\$20 (1,000 credits) + \$0.04/credit overage	N/A	New row.

Scenario	Min Monthly	Heavy Usage (Cloud)	Heavy Usage (Local)	Notes
AWS Kiro — Pro+	\$40/mo	\$40 (2,000 credits) + overage	N/A	New row.
AWS Kiro — Pro Max	\$100/mo	\$100 (5,000 credits) + overage	N/A	New row.
AWS Kiro — Power	\$200/mo	\$200 (10,000 credits) + overage	N/A	New row.
JetBrains AI — Free	\$0/mo	\$0 (3 credits/30 days)	N/A	Old doc had a single vague “\$24.90/mo All Products Pack” figure; that bundling is gone.
JetBrains AI — Pro	\$10/user/mo	\$10 (10 credits/mo) + \$1/credit top-up	N/A	New row.
JetBrains AI — Ultimate	\$30/user/mo	\$30 (35 credits/mo) + \$1/credit top-up	N/A	New row.
Tabnine — Code Assistant	\$39/user/mo (annual)	\$39/user/mo	N/A	Replaces old doc’s “~\$12/month Pro” — that tier was sunset in 2025; a correction, not inflation- adjustment.

Scenario	Min Monthly	Heavy Usage (Cloud)	Heavy Usage (Local)	Notes
Tabnine — Agentic Platform	\$59/user/mo (annual)	\$59/user/mo	N/A	New tier — adds MCP integration, autonomous workflows, CLI access.
Zed AI — Personal	\$0/mo	\$0 (2,000 edit predictions/mo)	N/A	Replaces old doc’s vague “free tier and credits system.”
Zed AI — Pro	\$10/mo	\$10 + token overage at API list price +10% (\$5 tokens included)	N/A	New precise figure.
Zed AI — Business	\$30/user/mo	\$30/user/mo	N/A	New tier.
OpenAI Codex CLI	Bundled in ChatGPT plan	Free (\$0), Go (\$8/mo), Plus (\$20/mo), Pro (\$100/mo) — token-based credits within plan; local --oss mode is \$0	\$0 via --oss (Ollama/LM Studio/MLX + GPT-OSS- 20B/120B)	New tool — major gap fill.

Scenario	Min Monthly	Heavy Usage (Cloud)	Heavy Usage (Local)	Notes
OpenHands	\$0 tool + API	Varies by chosen provider/model (e.g., ~\$25/mo on DeepSeek v4-flash, ~\$450-675/mo on Claude Sonnet 5)	N/A	Docker overhead is a hardware/time cost, not a monthly fee.

9.2.4 Cost Migration Strategy

Scenario 1: Keep Claude-quality results, accept higher costs Use Aider or Cline with direct Anthropic API. Cost: ~\$450/month for heavy usage at Sonnet 5’s introductory rate (~\$675/month once the standard rate applies from September 2026). This is the worst-case scenario financially — you lose the Max plan subsidy, though the gap is narrower than it was under 2025 Sonnet pricing.

Scenario 2: Reduce cost with model diversity Use Aider or OpenCode with OpenRouter, routing to DeepSeek V4 or Qwen3-Coder for routine tasks and Claude/GPT-5.4 for complex ones. Cost: roughly \$50-200/month depending on routing strategy. Significant savings, some quality tradeoff on simpler models.

Scenario 3: Go local for routine work, cloud for hard tasks Use Aider or Cline with Ollama (Qwen3.5-9B, or a larger Qwen3-Coder variant if VRAM allows) for day-to-day coding, escalate to cloud API for complex multi-file refactors. Cost: near zero for routine work, occasional API costs for hard tasks.

Scenario 4: Fixed-cost IDE subscription Move to Cursor (Pro \$20/month, or Pro+ \$60/month for higher usage) or Devin Desktop/Windsurf (Pro \$20/month, Max \$200/month). You lose terminal-native workflow but get predictable pricing. Less capable at agentic tasks than Claude Code for heavy use cases.

Scenario 5: Free-for-individuals alternative Gemini CLI’s consumer free tier is gone as of June 18, 2026 — it’s enterprise-only now, so it no longer belongs in a “free” scenario. Google’s own replacement recommendation for those consumer users is **Google Antigravity 2.0**, its new agent-first IDE (Editor View plus an async multi-agent “Manager Surface”),

which is free for individuals and model-agnostic (Gemini 3 Pro, Claude Sonnet 4.5, GPT-OSS). Treat this as lower-confidence than the other scenarios here — Antigravity’s pricing and feature details are less firmly sourced than the rest of this analysis, beyond “exists, free for individuals, model-agnostic.”

10 Recommendations

10.1 If you want the closest CLI experience to Claude Code

Primary: Aider — mature, reliable, git-native, OpenRouter-supported, local LLM support. Start here. The absence of MCP and hooks is a real limitation if you’ve invested heavily in those, but the core editing workflow is the most Claude Code-like of any open-source tool.

Secondary: OpenCode — MCP support and broad provider flexibility (75+ connected providers) in a terminal tool, now with a large user base (183k GitHub stars, 7.5M+ MAU). One caveat that changes its positioning: OpenCode broke with Anthropic in January 2026 — ChatGPT subscriptions now work natively, but using Claude through OpenCode requires your own Anthropic API key rather than any bundled access. If part of OpenCode’s appeal to you was Claude-adjacency, verify that still fits your setup.

Worth evaluating if you want mainstream backing or hybrid local/cloud: OpenAI’s Codex CLI has become a major CLI-native competitor in its own right (95.9k GitHub stars, bundled into ChatGPT plans from Free up to Enterprise, local-model support via `--oss` for Ollama/LM Studio/MLX) — it’s the closest thing to a first-party OpenAI answer to Claude Code. Sourcegraph’s Amp (the 2026 rebrand of Cody) offers a hybrid local-CLI-plus-cloud-“Orbs” model with pay-as-you-go, no-markup pricing for individuals, though its exact rate card isn’t published — treat it as promising but under-specified until pricing firms up.

10.2 If you want the best IDE-integrated alternative

Primary: Cline — model-agnostic, OpenRouter-supported, browser use, MCP-compatible, Plan/Act mode, and now available beyond VS Code (JetBrains, Cursor, Windsurf, Zed, Neovim, plus a CLI preview). The most capable open-source agentic IDE tool available. Note: Roo Code, a 2024 fork of Cline that grew its own multi-agent following, was archived in May 2026; most of its userbase returned to Cline, reinforcing Cline’s position here.

Secondary: Continue.dev — best-in-class local LLM support and deep IDE integration across VS Code and JetBrains simultaneously, and as of September 2025 it also ships a headless `cn` CLI for scriptable, non-interactive use — useful if you want IDE-first but occasionally need a scriptable escape hatch.

Commercial option: Cursor (Pro \$20/month, Pro+ \$60/month, Ultra \$200/month) — if you want the best polished commercial experience and are comfortable with a VS Code-derived workflow. Note the terminology shift: “Composer” is now the name of Cursor’s own in-house model (Composer 2.5, May 2026), not the agent mode — autonomous execution is called “Agent mode.” Cursor 3 supports up to 8 parallel isolated agents on separate branches, which brings it closer to Claude Code’s autonomy level for some workflows, though it remains OpenRouter- and local-model-incompatible.

10.3 If cost is the primary constraint

Free option: Gemini CLI’s consumer free tier ended June 18, 2026 — Google now serves individuals only through enterprise Code Assist tiers or steers them to **Google Antigravity 2.0**, its new free-for-individuals, model-agnostic agent IDE (Gemini 3 Pro, Claude Sonnet 4.5, GPT-OSS). If a free, no-strings option matters to you, Antigravity — not Gemini CLI — is the current answer, though it’s a newer, less battle-tested product than Gemini CLI was.

Ultra-cheap option: Aider or OpenCode with DeepSeek V4 Flash via OpenRouter — approximately \$25/month for heavy usage with a capable (if not Claude-class) model. (The older `deepseek-chat/deepseek-reasoner` models this recommendation used to point to are sunset July 24, 2026 — make sure any existing config points at the V4 line.)

Zero API cost: Any OpenRouter- or Ollama-compatible tool + Qwen3.5-9B locally — requires a GPU (16GB VRAM is now a realistic bar) but then costs nothing per query.

A caution on Devin: Devin’s headline “\$20/month” is easy to misread as a budget option. In practice it’s a \$20/month base plus pay-as-you-go ACU billing at \$2.25/ACU (roughly \$9/hour of autonomous work), and realistic individual moderate use runs **\$70-220/month all-in** — closer to a mid-tier Claude API budget than a cheap alternative. It’s not recommended here as a cost-constrained choice; it competes on autonomy/capability, not price.

10.4 If MCP server investments are critical

Tools with confirmed MCP support: Claude Code (native), OpenCode, Cline, Gemini CLI, Goose (MCP is a core architectural pillar, with 70+ extensions and 3,000+ compatible servers), OpenHands (config, multiple transports), and Amp. Of these, OpenCode and Cline remain the strongest alternatives for a daily coding workflow; Goose is worth a closer look than before given how central MCP now is to its design, and its co-stewardship (with Anthropic and OpenAI) of the Linux Foundation’s Agentic AI Foundation.

10.5 If you need extensibility similar to Claude Code’s skills+hooks

The honest answer: no alternative matches Claude Code’s skills+hooks system. The closest approximations: - **Skills equivalents:** `.clinerules` (Cline), `CLAUDE.md`-style files, custom system prompts in any tool - **Hooks equivalents:** Git hooks, shell wrappers, CI/CD tooling — not built into any alternative - **MCP equivalents:** Cline, OpenCode, Gemini CLI, Goose, OpenHands, and Amp all support MCP — your server investments are portable - **Instruction-file equivalents:** if per-tool rule files (rather than MCP servers) are your bigger migration cost, `AGENTS.md` is now read natively by 30+ tools and is a better consolidation target than any single vendor’s format

10.6 Migration strategy

Rather than a hard switch, a practical migration path:

1. **Set up Aider** as a Claude Code complement today. Get comfortable with it. It is free to try with Haiku (cheap) or Ollama (free).
2. **Test OpenCode** — its MCP support and provider flexibility make it a strong candidate for a full Claude Code replacement, with the caveat that Claude access now requires your own API key rather than any bundled arrangement.
3. **Reassess your free-tier fallback** — Gemini CLI’s consumer free tier is gone; Google Antigravity 2.0 is the closest current free-for-individuals substitute if cost-free work matters to you.
4. **Invest in OpenRouter** — get an API key. With OpenRouter, you’re never locked to a single model again. As model prices drop (historically, they do), your costs drop automatically.
5. **Protect your MCP investments** — build MCP servers in preference to tool-specific plugins wherever possible. MCP compatibility is growing across the ecosystem. **Do the same for AGENTS.md** if you maintain per-tool instruction/rule files (`.clinerules`, `.cursorrules`, `CLAUDE.md`) — it’s now read by 30+ tools and is a lower-maintenance way to keep project instructions portable than maintaining one file per vendor.

11 References

Primary sources (verified):

- OpenRouter works-with-openrouter page: openrouter.ai/works-with-openrouter
- Aider OpenRouter documentation: aider.chat/docs/llms/openrouter.html
- Continue.dev Ollama guide: docs.continue.dev/guides/ollama-guide

- OpenHands MCP SDK guide: docs.openhands.dev/sdk/guides/mcp
- OpenHands MCP settings: docs.openhands.dev/openhands/usage/settings/mcp-settings
- OpenHands GitHub (post org-migration): github.com/OpenHands/OpenHands
- OpenCode documentation: opencode.ai/docs/providers/
- OpenCode GitHub (new org, post Jan 2026 rebrand): github.com/anomalyco/opencode
- Artificial Analysis coding agents taxonomy: artificialanalysis.ai/agents/coding
- Awesome CLI Coding Agents (bradAGI): github.com/bradAGI/awesome-cli-coding-agents
- Claude Max plan / usage limits: support.claude.com/en/articles/11049741-what-is-the-max-plan (accessed 2026-07-06)
- Claude Max session-limit change: morphllm.com/claude-code-usage-limits (accessed 2026-07-06)
- Anthropic API pricing: platform.claude.com/docs/en/about-claude/pricing (accessed 2026-07-06)
- Anthropic model overview: platform.claude.com/docs/en/about-claude/models/overview (accessed 2026-07-06)
- OpenAI API pricing: developers.openai.com/api/docs/pricing (accessed 2026-07-06)
- Google Gemini API pricing: ai.google.dev/gemini-api/docs/pricing (accessed 2026-07-06)
- DeepSeek API pricing: api-docs.deepseek.com/quick_start/pricing (accessed 2026-07-06)
- Alibaba Qwen API platform: qwen.ai/apiplatform (accessed 2026-07-06)
- GitHub Copilot BYOK (VS Code): docs.github.com/en/copilot/how-tos/copilot-sdk/auth/byok (accessed 2026-07-06)
- GitHub Copilot BYOK GA announcement (VS Code, tier availability): code.visualstudio.com/blogs/2026/07/06/copilot-byok-ga (accessed 2026-07-06)
- Cognition acquisition of Windsurf / Devin Desktop rebrand: devin.ai/desktop (accessed 2026-07-06)
- Cognition/Windsurf acquisition terms (\$250M, Dec 2025): idlen.io/news/cognition-devin-25-billion (accessed 2026-07-06)
- Google Antigravity as Gemini CLI's consumer successor: theregister.com/ai-ml/2026/05/20/bye-bye-gemini-cli (accessed 2026-07-06)
- Goose Linux Foundation Agentic AI Foundation co-stewardship: knightli.com/en/2026/05/08/goose-linux-foundation-agentic-ai-foundation-co-stewardship (accessed 2026-07-06)

Note: No direct citable URL was found this refresh for the AGENTS.md standard itself (formalized August 2025, donated to the Linux Foundation's Agentic AI Foundation December 2025)

2025) — flagging as needing a follow-up citation pass rather than inventing one.

Secondary sources (used for background, not for verified claims):

- pinggy.io/blog/top_cli_based_ai_coding_agents/
- artificialanalysis.ai/agents/coding
- morphllm.com/comparisons/claude-code-alternatives
- techbuddies.io/2026/01/22/goose-vs-claude-code-...
- finout.io/blog/anthropic-api-pricing
- ksred.com/claude-code-pricing-guide-...